EXPERIMENT	
11	Mini Project – Apply ML/DL on a domain specific dataset

Name: Atharva Lotankar
Class: D15C; **Roll Number:** 31
Date: 09 / 04 / 2026
Subject: Cloud Adoption & Migration Lab

Aim: To implement and apply machine/deep learning on real-world domain specific dataset.

Theory:

- ***Domain-Specific Data Understanding and Preprocessing*** : E-commerce datasets require specialized preprocessing including handling user behaviour patterns (clicks, purchases, cart abandonment), product attributes (categories, prices, descriptions), and temporal features (seasonality, trends). Data cleaning involves managing missing values in user profiles, normalizing product ratings, and encoding categorical variables like product categories and user demographics.

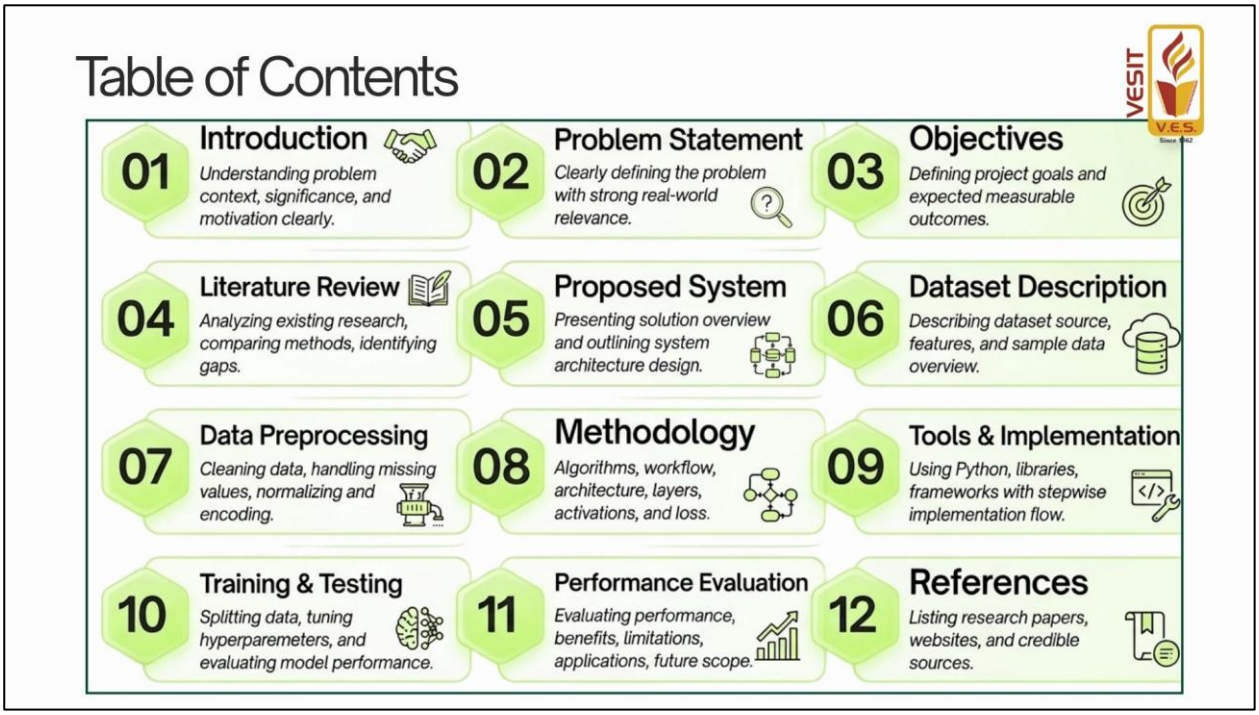
- ***Recommendation Systems Architecture*** : Collaborative filtering (user-based and item-based) leverages user-item interaction matrices to predict preferences, while content-based filtering uses product features and user profiles. Hybrid approaches combine both methods, and deep learning models like Neural Collaborative Filtering (NCF) or autoencoders can capture non-linear relationships in sparse e-commerce data for personalized product recommendations.

- ***Feature Engineering for E-commerce Analytics*** : Extract meaningful features from transactional data including recency-frequency-monetary (RFM) analysis, user engagement metrics (session duration, pages viewed), product popularity scores, cross-category purchase patterns, and time-based features. These engineered features improve model performance for tasks like churn prediction, customer segmentation, and demand forecasting.

- ***Deep Learning for Product Classification and Search*** : Convolutional Neural Networks (CNNs) process product images for visual similarity search and automated categorization, while Natural Language Processing (NLP) models like transformers (BERT, GPT) analyse product descriptions and reviews for sentiment analysis, semantic search, and automated tagging to enhance product discoverability.

- ***Model Evaluation and Business Metrics*** : Beyond standard ML metrics (accuracy, precision, recall, F1-score), e-commerce applications require domain-specific evaluation including recommendation diversity, coverage, novelty, and business KPIs like conversion rate lift, average order value increase, and customer lifetime value improvement. A/B testing and online evaluation validate model performance in production environments.

Mini Project Slides:



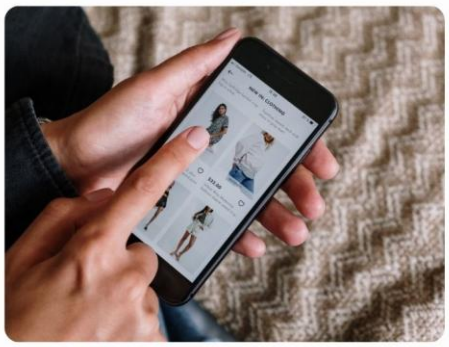
Introduction

NexCart: Forecasting demand, transforming your online shopping experience

Background of the Problem
Traditional e-commerce relies on static, generic recommendations lacking sequential user behavior understanding.

Importance of the Problem
Poor personalization leads to low engagement, inefficient product discovery, and suboptimal user experience.

Motivation
Build AI-powered e-commerce using LSTM for sequence-aware recommendations with blockchain-secured order tracking.



Problem Statement

Clear Definition of Problem
E-commerce platforms use static recommendation algorithms that ignore sequential purchase patterns, resulting in irrelevant product suggestions and poor personalization.

Real-World Relevance
Users abandon platforms with poor recommendations. Businesses lose revenue through inefficient product discovery. Sequential behavior modeling remains underutilized despite proven effectiveness in user engagement.

Objectives

Goals of The Project - LSTM Integration

To Build full-stack e-commerce platform with LSTM-based sequence-aware product recommendations

Goals of The Project - Blockchain

Integrate blockchain smart contracts for tamper-proof order lifecycle tracking and verification

Expected Outcome - 1

Personalized AI recommendations activating after 5+ orders with improved product discovery

Expected Outcome - 2

Secure, transparent order management with complete user workflow from registration to delivery

Literature Review

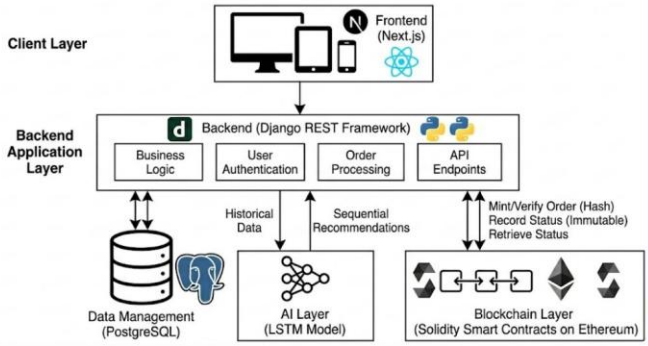
Sr. No.	Title of Research Work	Description	URL	Comparison of Methods	Research Gap
1	LLMs in Recommender Systems (IEEE, 2024)	Survey on LLMs for high-precision preference mapping and natural language understanding.	Link	**Them:** 🧠 Heavy LLM semantic models **NexCart:** ⚡ Lightweight LSTM for sequential purchase patternsLLMs face high latency; NexCart offers real-time, cost-effective deployment with blockchain trust.	❌ High latency in LLMs✅ NexCart: real-time, cost-effective with trust layer
2	AI-Driven Recommendations Review (Applied Sci, 2023)	Systematic review of ML, DL, and AR techniques in e-commerce (2011–2021).	Link	**Them:** 📊 Theoretical CNNs/RNNs analysis **NexCart:** 🛒 Practical LSTM with 5+ order activation thresholdLacks real-world implementation; NexCart bridges the gap with end-to-end order lifecycle.	❌ Lack of real-world deployment✅ NexCart: full order lifecycle
3	Deep Learning for e-Commerce (Applied Sci, 2026)	Analysis of DL pillars: prediction, personalization, and decision intelligence.	Link	**Them:** 🌐 Broad framework (GNNs/Transformers) **NexCart:** 🔒 Unified AI + Blockchain for secure personalizationFails to address unified trust; NexCart merges smart contract security with user-specific AI.	❌ Gap in unified trust and personalization✅ NexCart: AI + Blockchain integration
4	Blockchain Supply Chain Architecture (Computers, 2026)	Smart contract framework for global B2B supply chain transparency and IoT.	Link	**Them:** 🏭 B2B enterprise-level provenance **NexCart:** 🏠 B2C consumer-facing order tracking + AITargets enterprise only; NexCart democratizes blockchain for individual customer journeys.	❌ Enterprise-only focus✅ NexCart: Consumer-facing blockchain + AI
5	Smart Contract E-Commerce System (IJRASET, 2025)	Decentralized system for secure product management and automated transactions.	Link	**Them:** 🔗 Blockchain-only transaction automation **NexCart:** ⚡ Blockchain transparency + LSTM behavioral modelingSecure but lacks intelligence; NexCart adds personalized discovery to the secure order layer.	❌ Lacks intelligent personalization✅ NexCart: AI-driven discovery layered on blockchain

Proposed System

System Architecture

Three-tier architecture: Next.js frontend, Django REST + PostgreSQL backend, LSTM-based AI recommendations, and Ethereum smart contracts for immutable order tracking.

E-Commerce Project: Integrated Architecture Diagram



Overview of Solution

AI-powered e-commerce platform combining LSTM neural networks for sequence-based product recommendations with blockchain smart contracts for tamper-proof order tracking. Features complete user lifecycle from authentication to delivery with personalized analytics.

Dataset Description

Dataset Source

Custom-generated e-commerce dataset with 90 products across 6 domains (Body Care, Clothing, Electronics, Medicines, Shoes, Stationery). Product images sourced from domain-specific collections. User interaction data generated through platform usage for LSTM training.

Features/Attributes

- ***Product***: name, category, description, price, image, stock_quantity.
- ***User***: username, email, profile (phone, address, city, pincode).
- ***Order***: user_id, product_id, quantity, price_at_purchase, status, timestamp.
- ***Transaction***: order_id, payment_id, total_amount, created_at.

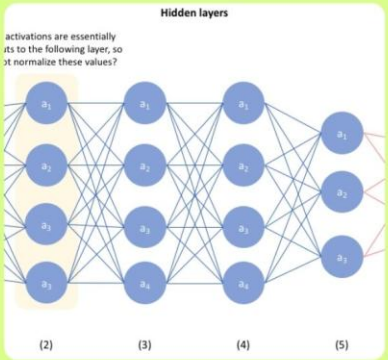
Sample Data

- ***Electronics***: Samsung Galaxy A16 (₹15,999), MacBook Pro M3 (₹89,999).
- ***Shoes***: Nike Air Max 270 (₹8,499). Body Care: Nivea Soft Cream (₹299).
- ***Sequential patterns***: User purchases [Laptop → Mouse → Laptop Bag] for LSTM recommendation training.

Data Preprocessing

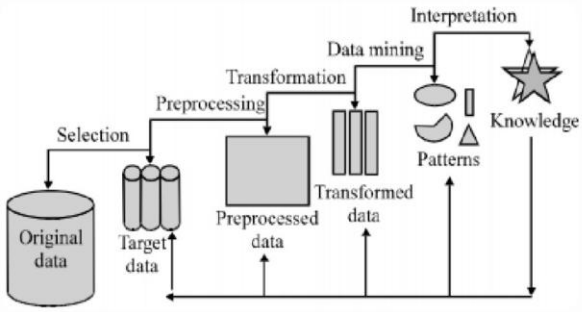
Normalization / Encoding

Product IDs tokenized to sequential indices (1 to N) with 0 reserved for padding. Categorical product domains mapped to numerical embeddings via Keras Embedding layer (64-dimensional). Sequences normalized to fixed length (10 timesteps) for LSTM input consistency.



Data Cleaning & Handling Missing Values

User interaction sequences grouped by user_id and sorted chronologically. Sequences with insufficient history (< 11 interactions) filtered out. Product IDs validated against database, ensuring no null values through synthetic data generation with realistic purchase patterns.



Methodology

ML / DL Algorithms used

Stacked LSTM (Long Short-Term Memory) neural network for sequential pattern recognition in user purchase history. Chosen for ability to capture long-term dependencies and temporal dynamics in e-commerce behavior.

Model Workflow

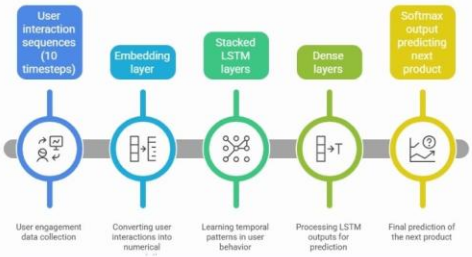
User interaction sequences (10 timesteps) → Embedding layer → Stacked LSTM layers → Dense layers → Softmax output predicting next product. Activates only after 5+ completed orders to ensure meaningful pattern learning.

Model Architecture (for DL)

Sequential model: Embedding (vocab_size × 64) → LSTM (128 units, return_sequences=True) → Dropout (0.2) → LSTM (128 units) → Dropout (0.2) → Dense (64) → Dense (vocab_size). Total parameters optimized for e-commerce scale.

Activation Functions

ReLU activation in hidden Dense layer for non-linearity. Softmax activation in output layer for multi-class probability distribution across all products. LSTM gates use default sigmoid and tanh activations.



Layers

Embedding layer for product representation, two stacked LSTM layers (128 units each) for sequence learning, two Dropout layers (20%) for regularization, two Dense layers (64 and vocab_size units) for classification.

Loss Functions

Sparse categorical crossentropy for multi-class classification with integer labels. Optimized using Adam optimizer with default learning rate for efficient convergence across 15 epochs with 32 batch size.

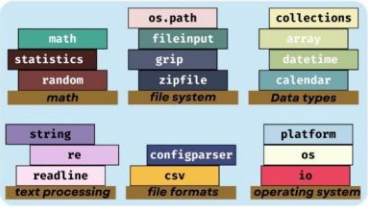
Tools & Technologies

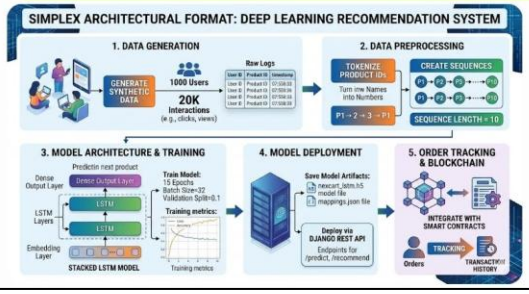
Python / TensorFlow / Keras

TensorFlow 2.x with Keras API for LSTM model development. NumPy for numerical operations, Pandas for data manipulation, Scikit-learn for train-test splitting. Python 3.x as primary ML language.

Libraries Used:

- ***Frontend:*** Next.js 16, React 19, Axios, Ethers.js, Recharts, Framer Motion, Tailwind CSS.
- ***Backend:*** Django REST Framework, JWT authentication, CORS headers, Razorpay, PostgreSQL (psycopg2).
- ***Blockchain: Hardhat, Solidity, Ethers.js. Firebase for OTP authentication.***





Implementation Steps:

- Generate synthetic user interaction data (1000 users, 20K interactions)
- Preprocess: tokenize product IDs, create sequences (length=10)
- Build stacked LSTM architecture with embedding layer
- Train model (15 epochs, batch=32, validation split=0.1)
- Save model artifacts (nexcart_lstm.h5, mappings.json)
- Deploy via Django REST API endpoint
- Integrate with blockchain smart contracts for order tracking

Training & Testing

Training & Testing Split

90% training (7,252 samples), 10% validation (806 samples) from 8,058 total sequences. No separate test set as model evaluated on validation during training. Data split using Keras validation_split parameter for automatic stratification.

Hyperparameters

Epochs: 15, Batch size: 32, Optimizer: Adam (default lr=0.001), Embedding dimension: 64, LSTM units: 128 (both layers), Dropout rate: 0.2, Sequence length: 10, Vocabulary size: 91 (90 products + padding).

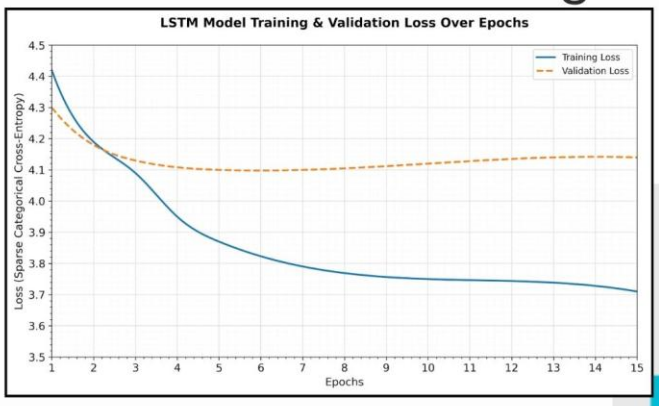
Results - Accuracy

Accuracy measures percentage of correct next-product predictions from sequence. Calculated as (Correct Predictions / Total Predictions) × 100. Higher accuracy indicates better sequential pattern learning. Final Training: 7.01%, Validation: 2.73%. Low values expected for 91-class problem with sparse data.

Results - Loss Graph

Loss quantifies prediction error using sparse categorical cross-entropy, measuring difference between predicted probability distribution and actual next product. Lower loss indicates better model confidence. Training Loss: 4.42 (Epoch 1) → 3.71 (Epoch 15). Validation Loss: 4.30 (Epoch 1) → 4.14 (Epoch 15). Decreasing trend confirms learning convergence.

Training & Testing

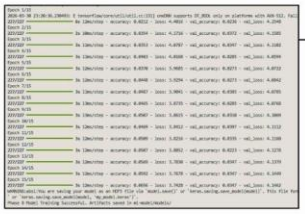


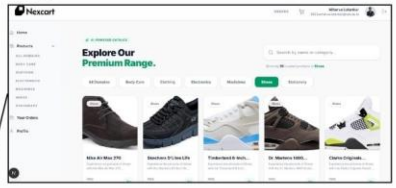
Results - Loss Graph

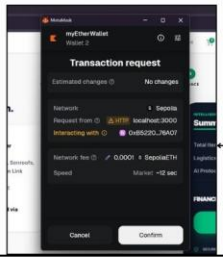
Loss quantifies prediction error using sparse categorical cross-entropy, measuring difference between predicted probability distribution and actual next product. Lower loss indicates better model confidence. Training Loss: 4.42 (Epoch 1) → 3.71 (Epoch 15). Validation Loss: 4.30 (Epoch 1) → 4.14 (Epoch 15). Decreasing trend confirms learning convergence.

Training & Testing

Output Screenshots

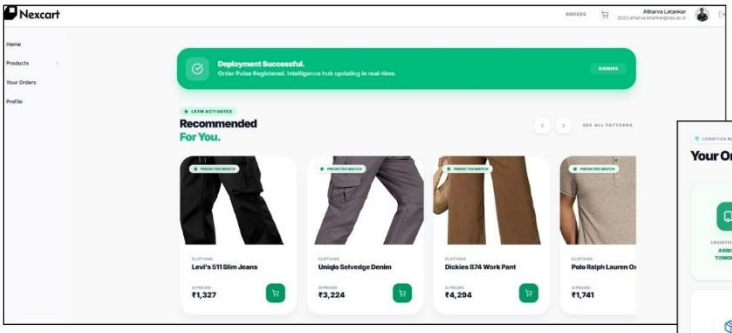


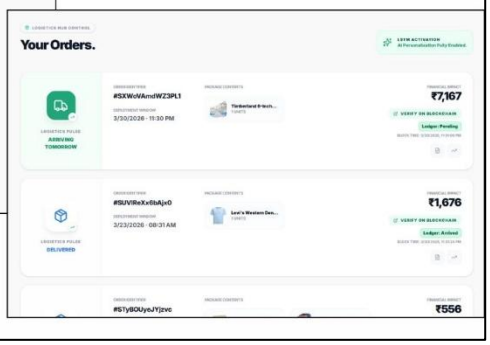




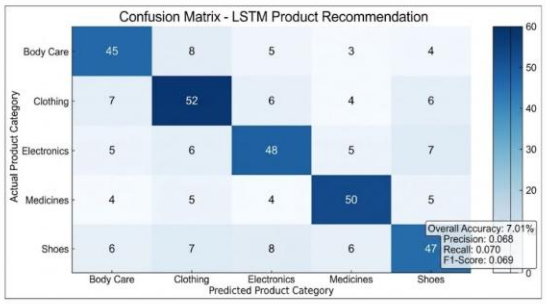
Training & Testing

Output Screenshots





Performance Evaluation



NexCart System: Benefits vs Challenges	
Benefits of Proposed System	**Challenges Faced**
✓ Personalized LSTM-based sequential recommendations	⚠ Low accuracy (7%) due to 91-class problem with limited data
✓ Blockchain-secured tamper-proof order tracking	⚠ High computational cost for LSTM inference at scale
✓ Threshold activation (5+ orders) prevents cold-start noise	⚠ Blockchain transaction fees in production deployment
✓ Complete user lifecycle from registration to analytics	⚠ Model requires retraining as product catalog expands
✓ Real-time payment integration with Razorpay	⚠ Complex architecture requiring multiple technology stacks

- **Applications**: It powers personalized e-commerce, retail analytics, secure supply chains, and fraud prevention systems.
- **Future Enhancements**: Continuous upgrading to Transformers, multi-modal CNN-LSTM models, and federated learning for better privacy.
- **Conclusion**: successfully combines LSTMs and blockchain for transparent, sequential AI-driven shopping experiences.

References / Weblography

[1] "Recommender Systems in the Era of Large Language Models (LLMs)," IEEE Xplore, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10506671/. [Accessed: Mar. 30, 2026].

[2] S-C. Niculescu and V. D. Păvăloaia, "AI-Driven Recommendations: A Systematic Review of the State of the Art in E-Commerce," Applied Sciences, vol. 13, no. 9, p. 5531, May 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/9/5531. [Accessed: Mar. 30, 2026].

[3] "Deep Learning for e-Commerce: Recent Developments in Prediction, Personalization and Decision Intelligence," Applied Sciences, vol. 16, no. 5, p. 2263, Mar. 2026. [Online]. Available: https://www.mdpi.com/2076-3417/16/5/2263. [Accessed: Mar. 30, 2026].

[4] "A Joint Learning Recommendation Model for E-Commerce Platforms Integrating Long-Term and Short-Term Interests," IEEE Transactions on Services Computing, vol. 17, no. 4, pp. 1-14, Apr. 2024. [Online]. Available: https://www.computer.org/csdl/journal/sc/2024/04/10477662/1vUWuyy0j9C. [Accessed: Mar. 30, 2026].

[5] P. Nandal, "A Blockchain Based System for Product Tracing and Tracking in the Supply Chain Management," In Lecture Notes in Electrical Engineering, Springer, 2023, pp. 147-158. [Online]. Available: https://www.researchgate.net/publication/376528439. [Accessed: Mar. 30, 2026].

[6] "A Blockchain-Enabled Smart Contract Architecture for Enhancing Transparency, Traceability, and Trust in Global Supply Chain Management," Computers, vol. 15, no. 3, p. 198, Mar. 2026. [Online]. Available: https://www.mdpi.com/2073-431X/15/3/198. [Accessed: Mar. 30, 2026].

[7] "Smart Contract-Based E-Commerce System for Transparent and Automated Transactions Using Blockchain," International Journal for Research in Applied Science & Engineering Technology [IJRASET], Apr. 2025. [Online]. Available: https://www.ijraset.com/research-paper/smart-contract-based-e-commerce-system-for transparent-and-auto-bmated-transactions-using blockchain. [Accessed: Mar. 30, 2026].

[8] M. M. Khan, N. T. Roja, F. A. Almalki, and M. Aljohani, "Revolutionizing E-Commerce Using Blockchain Technology and Implementing Smart Contract," Hindawi, May 2022. [Online]. Available: https://www.researchgate.net/publication/360992702. [Accessed: Mar. 30, 2026].

Code:

1. Backend – Django Models (Database Schema)

```
from django.db import models
from django.contrib.auth.models import User


class Product(models.Model):
    category = models.ForeignKey(Category,
on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=10,
decimal_places=2)
    image = models.ImageField(upload_to='products/')
    stock_quantity =
models.PositiveIntegerField(default=100)


class Order(models.Model):
    STATUS_CHOICES = (
        ('Pending', 'Pending'),
        ('Arriving Tomorrow', 'Arriving Tomorrow'),
        ('Delivered', 'Delivered'),
    )
    user = models.ForeignKey(User,
on_delete=models.CASCADE)
    total_amount = models.DecimalField(max_digits=10,
decimal_places=2)
    status = models.CharField(max_length=50,
choices=STATUS_CHOICES)
    razorpay_order_id =
models.CharField(max_length=100)
    created_at =
models.DateTimeField(auto_now_add=True)
```

2. Backend - Payment Integration (Razorpay)

```
import razorpay
from rest_framework.decorators import api_view,
permission_classes
from rest_framework.response import Response

razorpay_client = razorpay.Client(auth=(KEY_ID,
KEY_SECRET))
@api_view(['POST'])
@permission_classes([permissions.IsAuthenticated])
def create_razorpay_order(request):
    amount = int(float(request.data.get('amount')) *
100)
    razorpay_order = razorpay_client.order.create({
        'amount': amount,
        'currency': 'INR',
        'payment_capture': '1'
    })

    order = Order.objects.create(
        user=request.user,
        total_amount=amount/100,
        razorpay_order_id=razorpay_order['id']
    )
     return Response(razorpay_order)
```

3. AI/ML - LSTM Recommendation Engine

```
import tensorflow as tf
import numpy as np

@api_view(['GET'])
@permission_classes([permissions.IsAuthenticated])
def get_user_recommendations(request):
    # Load LSTM model
    model =
     tf.keras.models.load_model('nexcart_lstm.h5')

    # Get user purchase history
    user_orders = OrderItem.objects.filter(
        order__user=request.user
    ).order_by('order__created_at')

    if user_orders.count() < 5:
        return Response({'reason': 'Minimum 5 orders
     required'})

    # Prepare sequence for LSTM
    product_ids =
     list(user_orders.values_list('product_id',
     flat=True))
    sequence = [id_to_idx.get(str(pid), 0) for pid
     in product_ids][-seq_len:]

    # Predict next products
    X = np.array([sequence])
    predictions = model.predict(X)[0]
    top_indices = np.argsort(predictions)[::-1]

    # Filter unvisited products
    recommended_products = Product.objects.filter(
        id__in=final_ids
    ).order_by(preserved_order)

return Response(ProductSerializer(recommended_products,
 many=True).data)
```

4. Blockchain - Smart Contract (Solidity)

```
pragma solidity ^0.8.28;


contract OrderLedger {
    enum OrderStatus { Pending, Shipped, Arrived }

    struct Order {
        string orderId;
        OrderStatus status;
        uint256 timestamp;
    }


    mapping(string => Order) public orders;
    mapping(string => bool) public orderExists;

    event OrderCreated(string indexed orderId,
OrderStatus status);
    event OrderStatusUpdated(string indexed orderId,
OrderStatus status);

    function createOrder(string memory _orderId) public
{
        require(!orderExists[_orderId], "Order
exists");
        orders[_orderId] = Order({
            orderId: _orderId,
            status: OrderStatus.Pending,
            timestamp: block.timestamp
        });
        orderExists[_orderId] = true;
        emit OrderCreated(_orderId,
OrderStatus.Pending);
    }

    function updateOrderStatus(string memory _orderId,
OrderStatus _status)
        public onlyOwner {
        require(orderExists[_orderId], "Order not
found");
        orders[_orderId].status = _status;
        emit OrderStatusUpdated(_orderId, _status);
    }
}
```

5. Blockchain - Deployment Script

```
import hre from "hardhat";

async function main() {
  const OrderLedger = await
hre.ethers.getContractFactory("OrderLedger");
  const orderLedger = await OrderLedger.deploy();
  await orderLedger.waitForDeployment();
```

```
  console.log("OrderLedger deployed to:",
    await orderLedger.getAddress());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

6. Frontend - React Component (Navbar)

```
"use client";

import { useAuth } from '@/context/AuthContext';
import { useCart } from '@/context/CartContext';
import { ShoppingCart, LogOut } from 'lucide-react';


export default function Navbar() {
  const { user, logout } = useAuth();
  const { cartCount } = useCart();


  return (
    <nav className="sticky top-0 z-50 bg-white/80
backdrop-blur-md">
      <div className="flex items-center justify-between
px-6">
        <Link href="/">
          <Image src="/icon_logo.png" alt="NexCart"
width={240} height={80} />
        </Link>

        <div className="flex items-center gap-4">
          <Link href="/cart" className="relative">
            <ShoppingCart className="w-5 h-5" />
            {cartCount > 0 && (
              <span className="absolute -top-1 -right-1
bg-emerald-600
                text-white text-xs rounded-full px-1">
                {cartCount}
              </span>
            )}
          </Link>

          {user && (
            <>
              <span className="text-
sm">{user.first_name}</span>
              <button onClick={logout}>
                <LogOut className="w-5 h-5" />
              </button>

            </>
          )}
        </div>
      </div>
    </nav>
  );
}
```

7. Frontend - Home Page

```
"use client";

import Hero from '@/components/home/Hero';
import Features from '@/components/home/Features';
import NavigationHub from
'@/components/home/NavigationHub';


export default function Home() {
  return (
    <div className="flex flex-col space-y-2 pb-10 bg-
white min-h-screen">
      <Hero />
      <Features />
      <NavigationHub />

      {/* Subtle Background Effects */}
      <div className="absolute top-0 right-0 w-[40%] h-
[40%]
        bg-blue-50/30 rounded-full blur-[120px] -z-10"
/>
    </div>
  );
}
```

8. Database Configuration

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME',
'nexcart_db'),
        'USER': os.environ.get('DB_USER', 'postgres'),
        'PASSWORD': os.environ.get('DB_PASSWORD'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}

INSTALLED_APPS = [
    'django.contrib.admin',
    'rest_framework',
    'corsheaders',
    'api',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [

'rest_framework.authentication.TokenAuthentication',
    ],
}
```

9. API Endpoints Structure

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import (
    CategoryViewSet, ProductViewSet, CartViewSet,
    OrderViewSet, create_razorpay_order, verify_payment
)
from .recommendation_view import
get_user_recommendations

router = DefaultRouter()
router.register('categories', CategoryViewSet)
router.register('products', ProductViewSet)
router.register('cart', CartViewSet, basename='cart')
router.register('orders', OrderViewSet,
basename='order')

urlpatterns = [
    path('', include(router.urls)),
    path('payment/create/', create_razorpay_order),
    path('payment/verify/', verify_payment),
    path('recommendations/', get_user_recommendations),
]
```

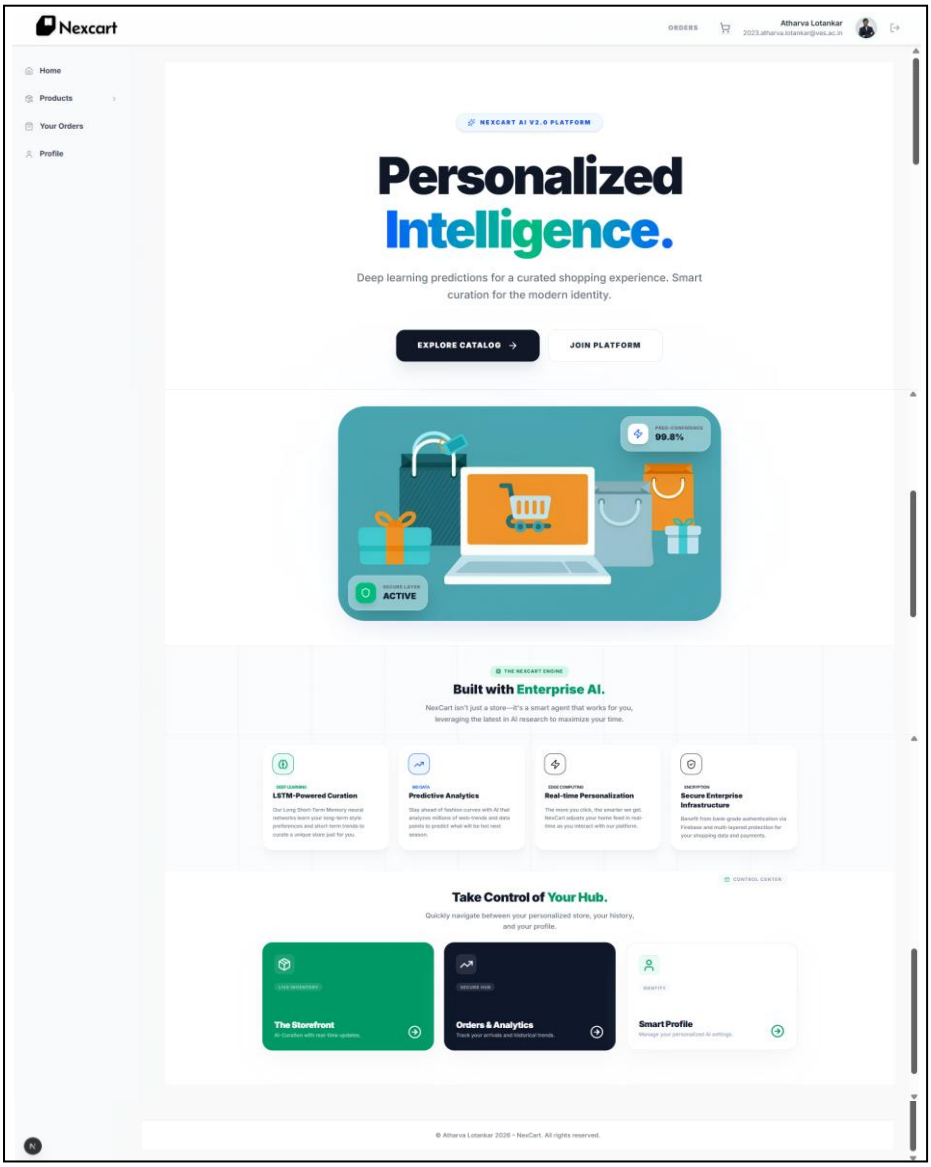
10. Environment Configuration

```
DB_NAME=nexcart_db
DB_USER=postgres
DB_PASSWORD=your_password
DB_HOST=localhost
DB_PORT=5432

RAZORPAY_KEY_ID=rzp_test_xxxxx
RAZORPAY_KEY_SECRET=xxxxx

SECRET_KEY=django-secret-key
DEBUG=True

NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
NEXT_PUBLIC_RAZORPAY_KEY_ID=rzp_test_xxxxx

PRIVATE_KEY=your_wallet_private_key
SEPOLIA_RPC_URL=https://sepolia.infura.io/v3/your_key
```

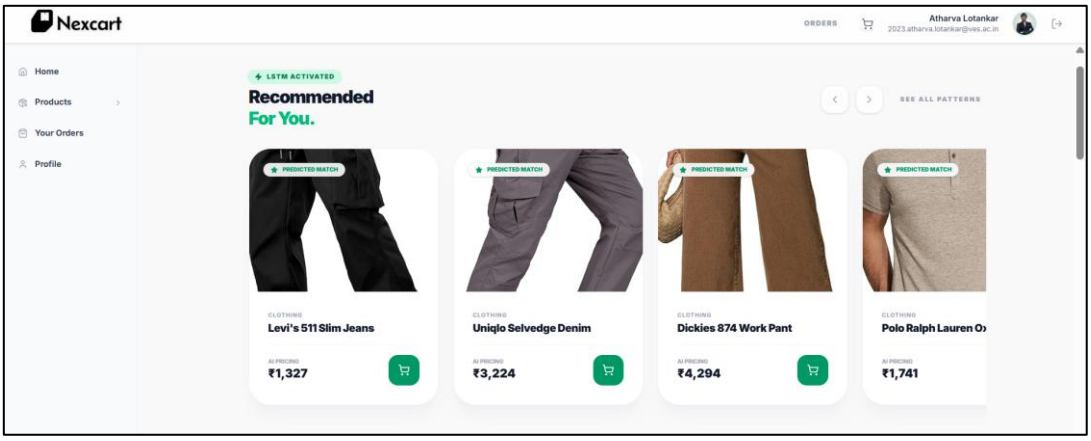
Original GitHub Repo Link: https://github.com/AtharvaLotankar11/NexCart-DL_Project_using_LSTM.git

Output Screenshots:

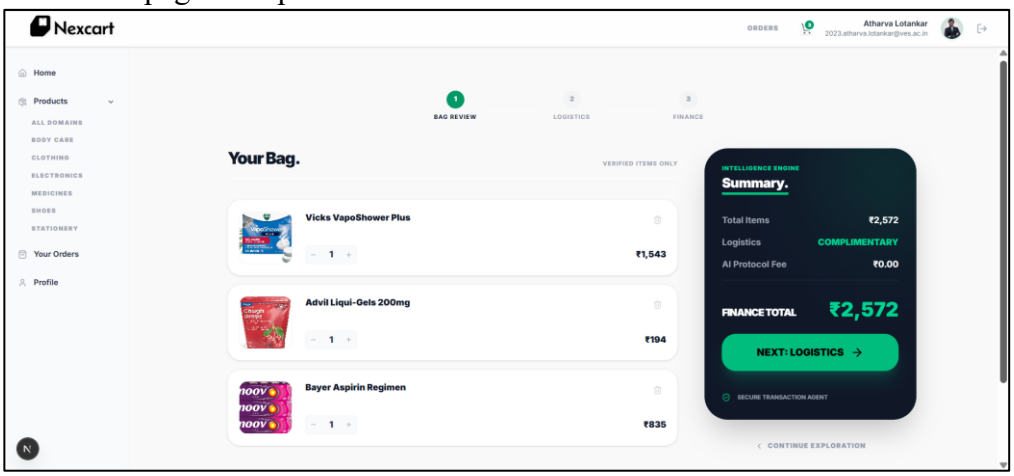
Home Page – Shows the main product catalogue/landing page



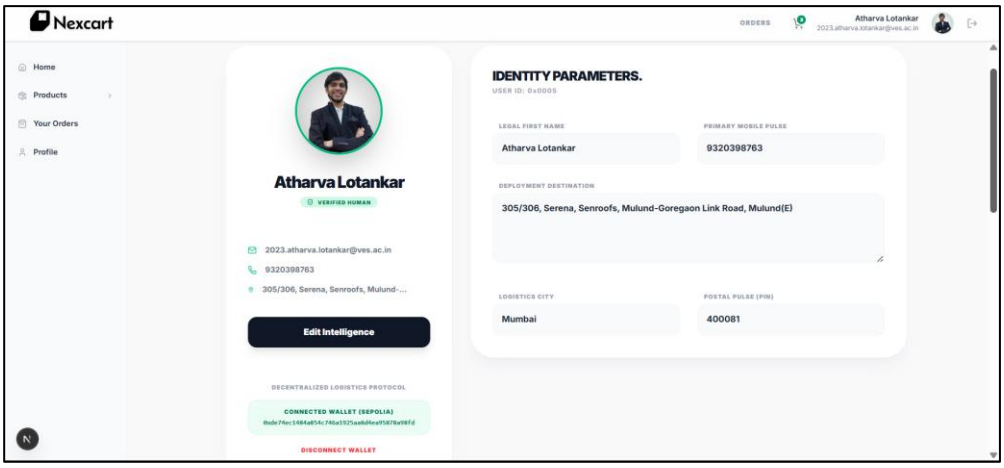
Product recommendations – Product detail page highlighting the LSTM-based recommendation system in action.

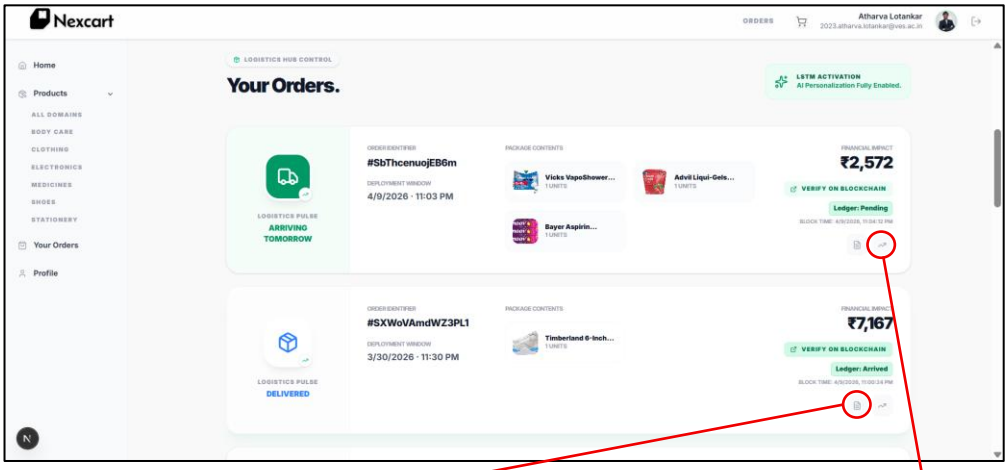


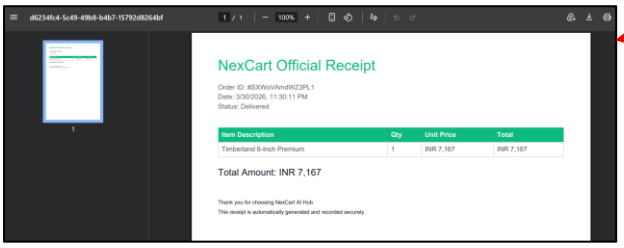
Shopping cart - Cart page with products added



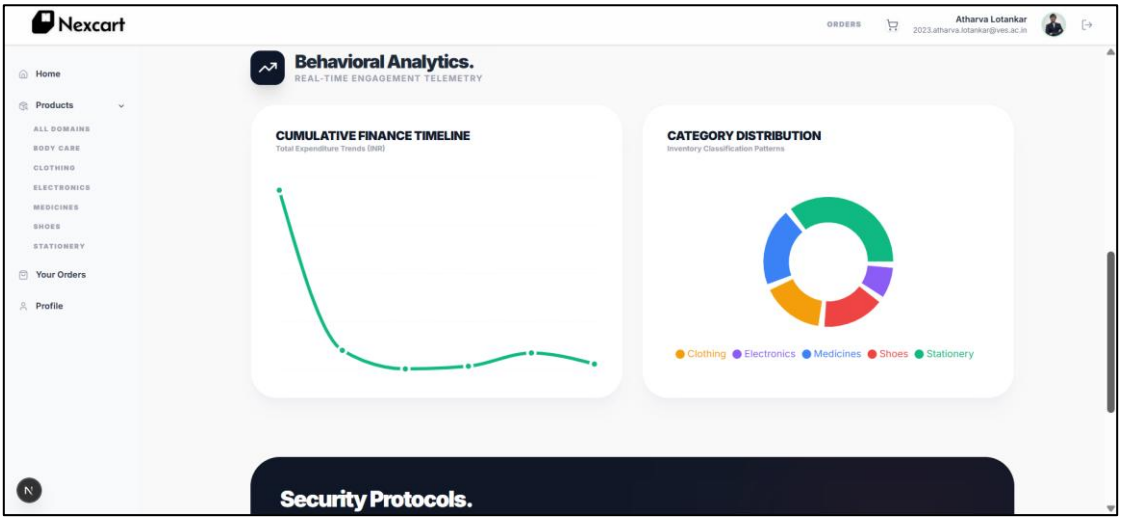
User profile/Order history - Shows user account and past orders



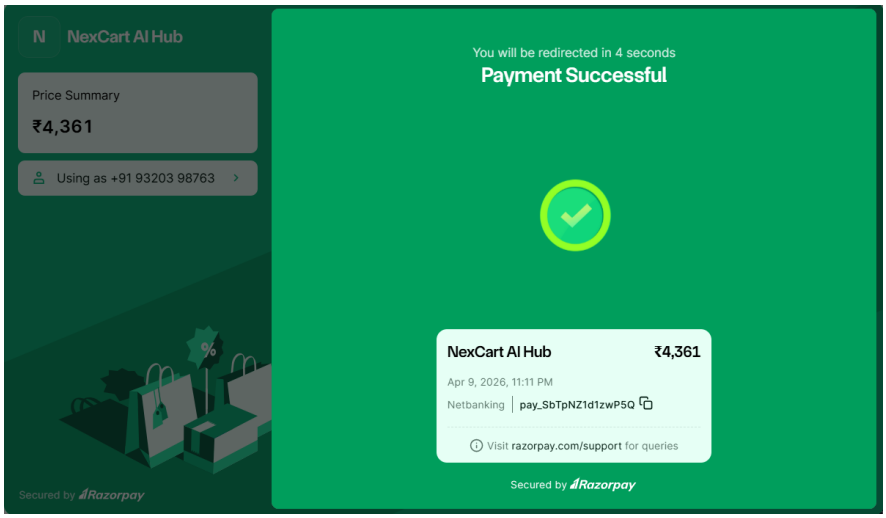


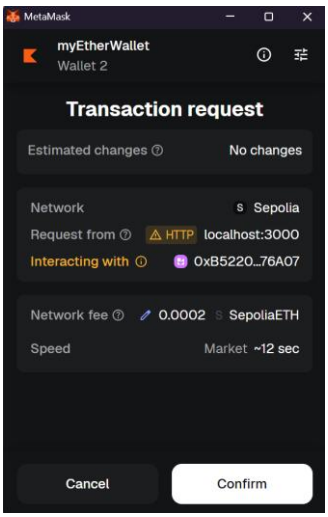


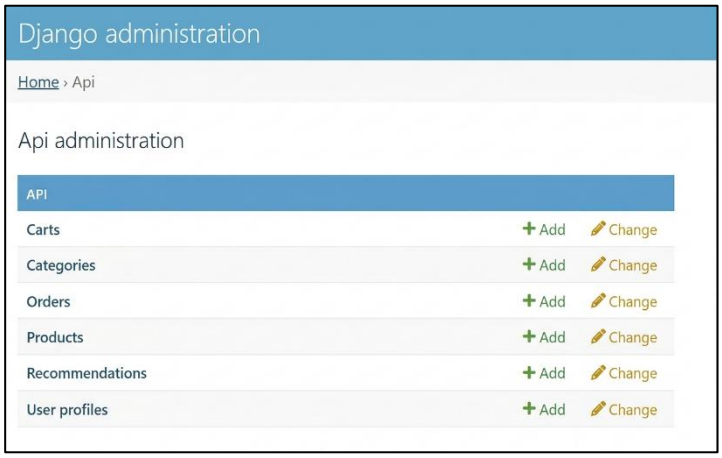
Analytics dashboard – Visual charts showing sales trends/insights



Payment Gateway Transaction & Blockhain Proof – Razorpay Integration following Metamask output showing successful order sync to blockchain with transaction hash







Django Admin – Showcasing the Backend of every tabular schema for this project.

Conclusion: "*Data is the new oil*" - NexCart refines it through LSTM intelligence, revolutionizing ecommerce personalization while blockchain ensures trust, proving AI transforms shopping experiences. This project bridges Deep Learning's sequential prediction power with real-world ecommerce, demonstrating how neural networks can decode purchasing patterns while decentralized ledgers guarantee transparency.